

EE675 Assignment 2

Naman Patidar (230679)

1.Part (a): Policy Network Design

We represent the policy $\pi_{\theta}(a | s)$ using a Multi-Layer Perceptron (MLP) implemented in PyTorch.

State Representation

The state consists of:

Predator position: (x_p, y_p)

Prey position: (x_{prey}, y_{prey})

Thus, the input dimension is **4**.

To improve training stability, the state values are normalized to the range $[0, 1]$ by dividing by $N - 1$.

Action Space

The predator can take **9 actions**:

- Stay, Up, Down, Left, Right
- Diagonal moves (Up-Left, Up-Right, Down-Left, Down-Right)

Thus, the output dimension is **9**.

Network Architecture

The policy network consists of:

- Input layer: 4 neurons
- Two hidden layers: 64 neurons each with ReLU activation
- Output layer: 9 neurons (action logits)

We output **logits instead of probabilities** for numerical stability. The softmax operation is handled internally by PyTorch's Categorical distribution.

Action Selection

An action is sampled using:

$$a \sim \pi_{\theta}(\cdot | s)$$

The log-probability is required for computing the policy gradient.

2. Part (b): Gradient Estimation using REINFORCE

We implement the REINFORCE algorithm to estimate the policy gradient.

Trajectory Generation

Starting from an initial state, we simulate an episode for a fixed number of steps. At each step:

1. Sample an action from the policy
2. Apply it to the environment
3. Observe reward and next state

We store:

1. Log-probabilities: $\log \pi_{\theta}(a_t | s_t)$
2. Rewards: r_t

Return Calculation

We compute the discounted return:

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

This is computed efficiently by iterating backward through the trajectory.

Variance Reduction

To reduce variance in gradient estimates, we normalize the returns:

$$G_t \leftarrow \frac{G_t - \mu}{\sigma}$$

This significantly stabilizes training.

Policy Gradient Objective

The REINFORCE objective is:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

We implement this by defining the loss:

$$L = - \sum_t \log \pi_{\theta}(a_t | s_t) \cdot G_t$$

PyTorch Implementation

- The score function $\nabla_{\theta} \log \pi_{\theta}(a | s)$ is obtained using **dist.log_prob(action)**
- Gradients are computed using:
loss.backward()
PyTorch's automatic differentiation computes the required gradients efficiently.

3. Part (c): Simple Stochastic Gradient Ascent (SGA)

We update the policy parameters using a manually implemented stochastic gradient ascent step.

Update Rule

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Since we minimize a negative objective in PyTorch, the update is implemented as:
`param -= learning_rate * param.grad.`

Learning Rate

We use:

$$\alpha = 0.005$$

Training Details

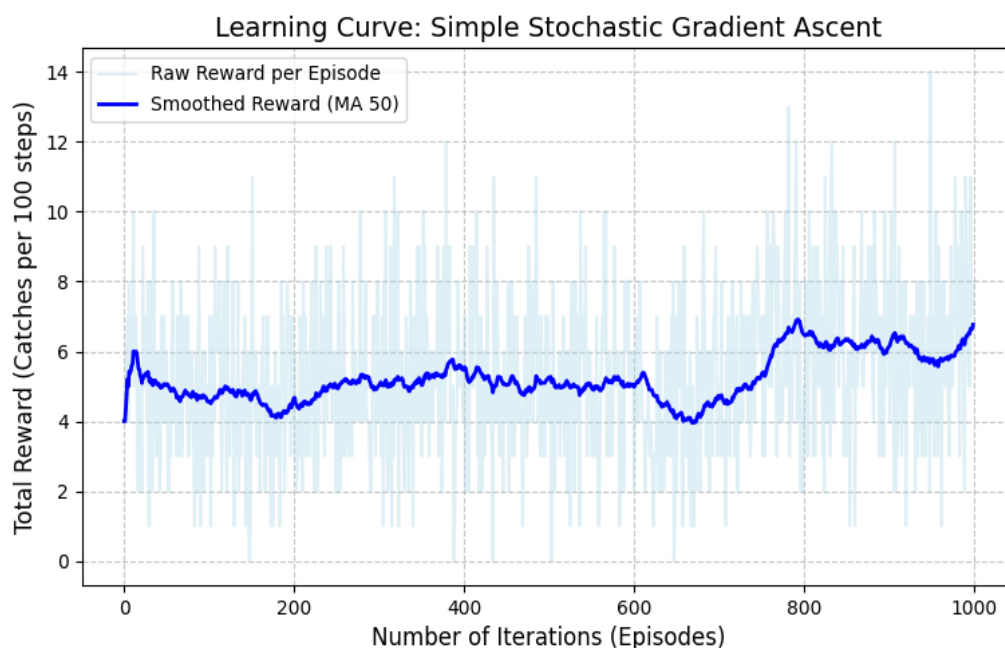
- Number of episodes: 1000
- Maximum steps per episode: 100
- Discount factor: $\gamma = 0.99$

Learning Curve

We plot:

- Total reward per episode
- A smoothed moving average (window = 50)

The curve shows a gradual increase in performance, though it remains noisy due to the stochastic nature of REINFORCE.



4. Part (d): Adam Optimizer

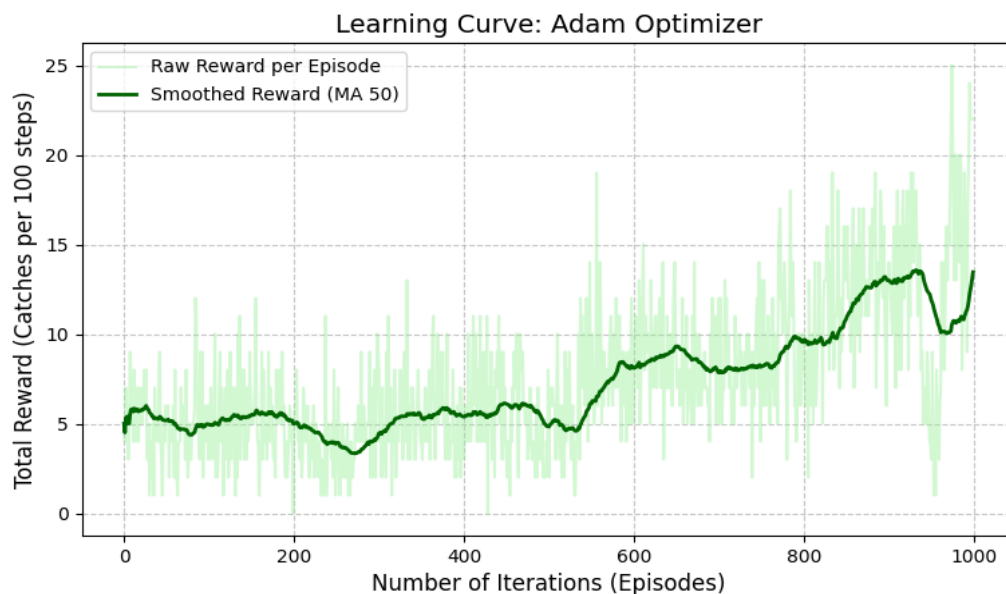
Instead of manually updating parameters, we use the Adam optimizer.

Adam improves training by:

- Using adaptive learning rates
- Maintaining momentum (first and second moments of gradients)
- Providing faster and more stable convergence

Comparison with SGA

- Adam converges faster
- Learning curve is smoother
- Less sensitive to learning rate



6. Observations

1. The policy successfully learns to increase the frequency of catching the prey.
2. Learning curves are noisy due to high variance in Monte Carlo estimates.
3. Return normalization significantly improves stability.

4. Adam performs better than simple SGA in terms of convergence speed and smoothness:-

The main issue with the REINFORCE algorithm is that it produces very noisy and unstable gradients. Since every episode depends on random actions and outcomes, the updates can vary a lot from one episode to another. Because of this, simple stochastic gradient ascent (SGA) ends up following every random fluctuation, which makes the learning process unstable and slow.

Adam helps fix this problem in two important ways.

1. It uses something like **momentum**. Instead of trusting just the current gradient, it keeps track of the average direction of past gradients. So, if one episode gives a strange or misleading update, Adam doesn't immediately follow it. It smooths things out and keeps the learning moving in a more consistent direction.

2. Adam adjusts the learning speed for each parameter separately. In SGA, every weight is updated with the same learning rate, no matter how unstable it is. But in Adam, if a particular weight is seeing large or erratic gradients, the optimizer automatically reduces its step size. This prevents big, harmful updates and makes training more stable.

In short, SGA is easily affected by randomness in the environment and tends to jump around a lot, while Adam is more controlled. It smooths out the noise and helps the model improve in a steady and reliable way.

7. Conclusion

The REINFORCE algorithm provides an unbiased estimate of the gradient but suffers from high variance, which can be mitigated using normalization and better optimizers like Adam.